AI COLLEGE ENQUIRY CHATBOT

Project Report

Submitted by	Sanika B.S.
USN	4MN24CI044
College	Maharaja Institute of Technology, Thandavapura
Department	Computer Science and Engineering (AI & ML)
Project Guide	Not specified
Academic Year	Not specified

A prototype demonstrating natural-language college enquiry handling using NLP, machine learning and a Flask web application.

Abstract

The AI College Enquiry Chatbot is a web-based prototype designed to answer common college-related questions through a conversational interface. It applies text preprocessing, TF-IDF feature extraction and a Logistic Regression classifier to identify the intent of a user's question. A Flask backend connects the machine-learning pipeline to a browser-based chat interface. The prototype covers 13 enquiry intents, including admissions, courses, fees, eligibility, facilities, internships and placements. The project demonstrates how natural language processing can make frequently requested information easier to access. The current dataset is self-created and uses illustrative sample-institute information; it must be replaced with verified institutional content before official use.

1. Introduction

Students regularly seek information about admission procedures, available courses, eligibility, fees, scholarships, examinations, hostel facilities, library services, internships and placements. When information is distributed across multiple pages or handled manually, finding a quick answer can be inconvenient. A chatbot provides a single conversational interface through which users can submit questions in everyday language.

This project develops a prototype that predicts the intent of a question and returns a corresponding predefined response. It combines a machine-learning model with a lightweight web application.

2. Problem Statement

Traditional college enquiry methods may require students to browse multiple pages or contact staff for routine information. The objective is to develop a web-based chatbot that can understand common enquiry patterns, classify them into predefined categories and return relevant responses with a fallback for unsupported questions.

3. Objectives

• Develop a simple and accessible browser-based chatbot interface.

• Create a labelled FAQ dataset for common college enquiry categories.

• Apply text preprocessing and TF-IDF feature extraction.

• Train a Logistic Regression model for intent classification.

• Integrate the model with a Flask backend and test end-to-end behaviour.

• Provide a fallback response when the model is not sufficiently confident.

4. Scope of the Project

The current dataset contains 299 self-created question examples across 13 intents: academic calendar, admission, contact, courses, eligibility, examinations, fees, greeting, hostel, internships, library, placements and scholarships. The system is a prototype for common FAQ-style enquiries. It does not currently retrieve live information from college databases or official websites.

5. Existing and Proposed System

Existing system: Students may depend on manual enquiries, static FAQ pages or searches across separate website sections. These approaches can involve repeated effort and may not respond to different phrasings of the same question.

Proposed system: The user enters a question in a chatbot interface. The Flask application passes the text to the prediction pipeline, which predicts an intent and selects a mapped answer. If confidence is below the configured threshold, the chatbot returns a fallback response.

6. System Requirements

Hardware requirements

• A computer or laptop capable of running Python and a modern web browser.

• Internet access is required for installation of dependencies; the local prototype can run on the same machine.

Software requirements

• Python 3.9 or later.

• Flask, scikit-learn, pandas, NumPy and Joblib.

• HTML, CSS and JavaScript for the frontend.

• A modern web browser and a code editor or terminal.

7. System Architecture

The architecture consists of five main stages. First, the user submits a question through the browser interface. Second, the Flask backend receives the message. Third, preprocessing normalizes the text. Fourth, the TF-IDF vectorizer converts the text into numerical features and Logistic Regression predicts an intent. Finally, the response layer returns the answer associated with that intent or a fallback when confidence is insufficient.

8. Methodology

• **Dataset preparation:** Create and label FAQ-style questions under the 13 supported intents.

• **Text preprocessing:** Normalize user input and prepare it for vectorization.

• **Feature extraction:** Use word n-grams (1–2) and character n-grams (2–4) with TF-IDF.

• **Model training:** Train a Logistic Regression classifier on labelled examples.

• **Evaluation:** Use a stratified 80/20 train/test split and review additional manually authored questions.

• **Application integration:** Connect prediction and response selection to the Flask interface.

9. Implementation Details

The project separates the web application, chatbot logic, preprocessing and model training into different Python modules. app.py runs the Flask application. chatbot.py handles intent prediction and response selection. preprocessing.py prepares input text. train_model.py trains and evaluates the classifier. The repository also includes folders for the dataset, trained models, templates, static files, tests and reports.

A confidence threshold of 0.35 is configurable. When the model's confidence is below this value, the chatbot uses a fallback rather than returning a potentially unreliable answer.

10. Testing and Results

The documented dataset contains 299 examples. A stratified split uses 239 examples for training and 60 for testing. The reported evaluation results are 90% accuracy, weighted precision of 0.9254, weighted recall of 0.90 and weighted F1-score of 0.9007. A separate manually authored test set recorded 37 successful cases out of 38.

These results should be treated as preliminary. The dataset is self-created, the held-out test set is relatively small, and the model is ultimately refitted on the full dataset. Additional evaluation with unseen questions from real students is necessary before making strong claims about real-world performance.

11. Limitations

• The FAQ content uses a fictional Sample Institute of Technology (SIT); details are illustrative and are not verified information about Maharaja Institute of Technology, Thandavapura.

• The model is limited to the intents and phrasings represented in its training dataset.

• Unseen wording, spelling errors, ambiguous questions and multi-part questions may lead to incorrect predictions.

• The prototype does not currently connect to a live college database or official information source.

12. Future Enhancements

• Replace sample data with verified and approved college information.

• Add multilingual support and improve handling of spelling variations and paraphrases.

• Expand the dataset and evaluate with a larger, independently collected test set.

• Explore semantic embeddings or transformer-based intent classification.

• Add an administrator interface for updating FAQs and analytics for unanswered questions.

• Deploy the application to a secure hosting platform for wider access.

13. Conclusion

The AI College Enquiry Chatbot demonstrates an end-to-end application of natural language processing and machine learning to a common student-support task. TF-IDF features and Logistic Regression are used to classify questions into predefined enquiry intents, while Flask provides the web interface and backend integration. The prototype provides a foundation for a more useful college enquiry assistant, provided that its sample content is replaced with verified institutional information and its performance is validated through broader testing.

14. Instructions to Run the Project

• Create and activate a Python virtual environment.

• Install dependencies using: pip install -r requirements.txt

• If required, build the dataset using: python dataset/build_dataset.py

• Train the model using: python train_model.py

• Start the application using: python app.py

• Open http://localhost:5000 in a browser.

• Run tests using: python -m unittest tests.test_chatbot -v

15. References

• Scikit-learn documentation: https://scikit-learn.org/

• Flask documentation: https://flask.palletsprojects.com/

• Python documentation: https://docs.python.org/3/

• Project source code and README in the associated GitHub repository.

Appendix: Data Accuracy Notice

The current FAQ dataset contains fictional sample-institute information. Any sample fee amounts, dates, contact details, course details or facility descriptions must not be presented as official college facts. Before demonstrating the system as a college-specific assistant, replace the sample entries with information verified from authorized college sources.

Student: Sanika B.S. **USN:** 4MN24CI044

College: Maharaja Institute of Technology, Thandavapura

Project Guide: Not specified **Academic Year:** Not specified